

Monadic Constraint Programming with Gecode

Pieter Wuille and Tom Schrijvers*

Department of Computer Science, K.U.Leuven, Belgium
FirstName.LastName@cs.kuleuven.be

Abstract. This paper presents FD-MCP, a finite domain modeling language on top of the Monadic Constraint Programming framework for Haskell. FD-MCP leverages Haskell’s rich static type system and powerful abstraction mechanisms for implementing syntactic sugar, model transformations and compilation to solver backends. Two backends are provided: a basic Haskell solver and a Gecode code generator. Our benchmarks establish that FD-MCP models are much more concise than hand-coded Gecode programs, more so when using disjunction, without sacrificing performance.

1 Introduction

High-level CP languages are an active topic of research. We can subdivide existing languages in two classes:

- Modeling languages: stand-alone languages whose programs are mapped to one or more constraint solver backends. They are usually highly declarative and allow convenient model transformations. Because designing and developing a new (modeling) language from scratch is a significant effort, existing modeling languages often run behind on the state of the art in programming language features such as static typing or abstraction mechanisms. They are not always Turing-complete.¹
- Programming languages with an API for writing and solving constraint models that is tied to a particular constraint solver. This API can be an integral part of the language (an embedded domain-specific language, or EDSL), or a set of library functions—or a combination of both. They are often Turing-complete and offer the abstraction mechanisms of the host language, but sometimes more imperative in nature. More often, a single model cannot be reused with different constraint solvers or transformed.

The FD-MCP framework aims to combine the advantages of both. It is a highly declarative modeling interface for finite domain (FD) problems, that supports model transformations and multiple backends for solving. It is implemented

* Post-Doctoral Researcher of the Research Foundation–Flanders (FWO-Vlaanderen).

¹ Although this may have the advantage of guaranteeing termination.

as an EDSL in Haskell [7], a state-of-the-art functional programming language that provides strong static typing and advanced abstraction mechanisms. FD-MCP extends the general Monadic Constraint Programming (MCP) [8] framework with FD-specific infrastructure.

One of FD-MCP’s backends is Gecode [5], an efficient, open, free and portable example of a constraint solving API for an existing language, namely C++. Due to its imperative and low-level nature, it is however more suitable as a backend for a modeling language than a direct implementation language.

The current state of our work in progress consists of:

- the FD-MCP modeling language embedded in Haskell,
- two backends for FD-MCP: a basic Haskell solver and the Gecode C++ library, and
- a compilation scheme to compile disjunctive FD models to Gecode, which provides no high-level support for disjunction,
- benchmarks that illustrate the succinctness of FD-MCP wrt. Gecode without sacrificing performance.

A more extensive report is available [10], as well as the source code.²

2 Monadic Constraint Programming

The MCP framework is a highly generic constraint programming framework for Haskell. It provides abstractions for writing constraint models, constraint solvers and search strategies. This paper focuses on the solving and modeling parts.

MCP defines type classes, Haskell’s form of interfaces, for **Solver**s and **Term**s:

```
class Monad s => Solver s where
  type Constraint s :: *
  add :: Constraint s -> s Bool
  run :: s a -> a
  ...

class Solver s => Term s t where
  newvar :: s t
```

A type that implements the **Solver** type class must provide a type to represent its constraints, an **add** function for adding constraints, a **run** function to extract the results, ... A solver type *s* must also be a monad [9]. A monadic value *s a* is an abstraction of a form of computation *s* that yields a result *a*. Constraint solvers are typically computations that thread an implicit state: the constraint store.

A solver also provides one or more types of terms: **Term s t** expresses that *t* is a term type of solver type *s*. Each term type provides a method **newvar** to generate new constraint variables of that type.

MCP also defines a data type **Model**, representing a model tree:

² <http://www.cs.kuleuven.be/~pieterw/site/Topics/FDMCP>

```

data Model s a
  = Return a                -- return a value
  | Add (Constraint s) (Model s a) -- add a constraint
  | Try (Model s a) (Model s a)   -- disjunction
  | Term s t => NewVar (t -> Model s a) -- introduce new variable
  | Dynamic (s (Model s a))      -- generate subtree

```

The model tree is parametrized in the constraint solver `s` and returned result type `a`. This provides a type-safe way for representing constraint problem models for arbitrary solvers and result types.

On top of the model data type, MCP provides syntactic sugar (functions that construct model trees), such as `exists` (create a variable), `exist n` (create a list of `n` variables), `addC` (add a constraint), `/\` (conjunction), `\/` (disjunction), `conj` (conjunction of list of models), ... Finally, `Model s` is also a monad.

3 The FD-MCP Framework

The FD-MCP framework introduces an extra layer of abstraction between the more generic `Solver` interface of the MCP framework and the concrete solver implementations.

In contrast to MCP's generic `Solver` interface, which is parametric in the constraint domain, the `FDSolver` interface of FD-MCP is fully aware of the finite domain (FD) constraint domain: both its syntax (terms and constraints) and meaning (constraint theory). It does however make abstraction of the particular FD solver and e.g. propagation techniques used. Hence, it provides a uniform modeling language that abstracts from the syntactic differences between different FD solvers.

On the one hand, this allows the development of solver-independent models, model transformations (e.g. for optimization) and model abstractions (capturing frequently used patterns). On the other hand, specific solvers may focus on the efficient processing of their constraint primitives without worrying about modeling infrastructure.

3.1 The Modeling Frontend

Figure 1 illustrates the FD-MCP modeling language with a model for the classical 7-11 puzzle. This puzzle concerns 4 unknown amounts whose sum and product are both 7.11. The amounts are represented in units of 1 cent.³

Line 1 starts the declaration of an FD-MCP model tree `model`. `exist` requests four new variables, and has an (anonymous) callback function for handling those new variables as parameter. This function itself takes the `list` of new variables

³ The `$` operator is function application (`f $ x = f x`) which associates to the right instead of the left. For instance, `f $ g $ h x` is equivalent to `f (g (h x))`, but syntactically more convenient.

```

1 model = exist 4 $ \list@[a,b,c,d] -> do
2   list 'allin' (0,711)
3   a + b + c + d @= 711
4   a * b * c * d @= 711000000
5   a @>= b
6   b @>= c
7   c @>= d

```

Fig. 1. Grocery example in MCP

as argument, individually aliased as **a**, **b**, **c** and **d**. Lines 2–7 define a conjunction of constraints that must hold for these variables:

- Line 2: restrict all variables’ domains to $0 \dots 711$.
- Line 3: the sum of the four variables is 711.
- Line 4: the product of the four variables is 711000000.
- Line 5–7: each variable is larger than or equal to the next one (to break symmetry).

FD-MCP Modeling Primitives The FD-MCP modeling language is built as a wrapper on top of the MCP solver interface. This way, the domain-independent combinators of the MCP framework, such as conjunction ($\wedge$) and existential quantification **exist** are available for FD models. The FD-MCP modeling language adds FD-specific constructs to that. Advanced FD constructs are defined in terms of a small set of core primitives, resulting in a layered structure.

The core constraints and terms are defined by the **FDConstraint** and **FDEExpr** types respectively:

```

data FDConstraint s
= Less (FDEExpr s) (FDEExpr s) -- [[Less x y]] ≡ [[x]] < [[y]]
| Diff (FDEExpr s) (FDEExpr s) -- [[Diff x y]] ≡ [[x]] ≠ [[y]]
| Same (FDEExpr s) (FDEExpr s) -- [[Same x y]] ≡ [[x]] = [[y]]
| Dom (FDEExpr s) Int Int      -- [[Dom x y z]] ≡ [[x]] ∈ {y, ..., z}
| AllDiff [FDEExpr s]          -- [[AllDiff [x1, ..., xn]]] ≡ ∧i≠j xi ≠ xj
| ...

data FDEExpr s
= Var (FDTerm s)              -- [[Var v]] ≡ [[v]]
| Const Int                   -- [[Const n]] ≡ n
| Plus (FDEExpr s) (FDEExpr s) -- [[Plus x y]] ≡ [[x]] + [[y]]
| Minus (FDEExpr s) (FDEExpr s) -- [[Minus x y]] ≡ [[x]] − [[y]]
| Mult (FDEExpr s) (FDEExpr s) -- [[Mult x y]] ≡ [[x]] * [[y]]
| ...

```

where the comment after each constructor shows its denotation.

Syntactic sugar On top of the core primitives, a number of convenient abstractions and syntactic sugar exist. Firstly, standard arithmetic operators and integer literals can be used for `FDEExpr s` thanks to an implementation of Haskell’s `Num` type class:

```
instance Num (FDEExpr s) where
  fromInteger = Const
  (+)         = Plus
  ...
```

Thus `Plus x (Mult (Const 2) y)` can be written succinctly as `x + 2 * y`. More syntactic sugar exists for writing constraints:⁴

```
x @< y      = Add (Less x y) true
x @> y      = y @< x
x @>= y     = x + 1 @> y
x @: (l,u)  = Dom x l u
xs 'allin' d = conj [ x @: d | x <- xs ]
...
```

The constraint operators are in fact a little smarter than that. Namely, the first one is defined as:

```
x @< y = Add (Less (simplify x) (simplify y)) true

simplify (Plus x (Const 0)) = simplify x
simplify ...                = ...
```

i.e. `@<` acts as a smart constructor that first simplifies its arguments. Argument simplification is a solver-independent model transformation built into our framework. It heuristically minimizes the number of variable occurrences and the number of nodes in the expression tree. Of course, programmers can define their own libraries of abstractions and model transformations in entirely the same style.

Example 1. Figure 2 lists the model obtained from Fig. 1 after desugaring the abstractions. The model consists of a first **NewVar** node, which has an anonymous function as parameter (after the \$). When passed a variable (of type `FDTerm s`, assuming the underlying solver is `s`), this function evaluates to a new **NewVar** node, again having a function as parameter. This pattern repeats itself four times, corresponding to the four variables introduced. The parameter of the fourth-level **NewVar** evaluates to a tree of **Add** nodes, each having an **FDConstraint** as one parameter, and the rest of the tree as child node. First four **Dom** constraints are added (corresponding to the `allin` in the model source code), followed by two **Same** constraints (corresponding to the two `@=` constraints), and finally three **Less** constraints. The **Return** node returns unit `()`, i.e., no value of interest.

⁴ The `[ x @: d | x <- xs ]` expression is a list comprehension, meaning “a list of `x @: d`, for each `x` from the list `xs`”.

```

NewVar $ \a ->
  NewVar $ \b ->
    NewVar $ \c ->
      NewVar $ \d ->
        Add (Dom a 0 711) $
          Add (Dom b 0 711) $
            Add (Dom c 0 711) $
              Add (Dom d 0 711) $
                Add (Same (Plus a (Plus b (Plus c d))) (Const 711)) $
                  Add (Same (Mult a (Mult b (Mult c d))) (Const 711000000)) $
                    Return ()

```

Fig. 2. FD model tree of the grocery example (see Figure 1)

3.2 Mapping to the solver backend

The backend takes care of compiling an FD-MCP model to a particular FD solver. To enable this compilation the solver must implement the following `FDSolver` type class (in addition to implementing the `Solver` type class of the MCP framework):

```

class Term s (FDTerm s) => FDSolver s where
  type FDTerm s :: *
  compile_constraint :: FDConstraint s -> Model s Bool

```

The `FDSolver` type class makes two demands of a solver `s`:

- It must provide a type `FDTerm s` for its terms.
- The function `compile_constraint` must take care of converting from an individual FD-MCP constraint to a model for the solver. Note that, to allow mapping a single FD-MCP constraint to a conjunction of solver constraints involving auxiliary variables, this function returns a model rather than a single constraint. This model is not allowed to contain any disjunctions.

The following law specifies the `compile_constraint` function:

Definition 1 (Denotation Preservation). *The `compile_constraint` function preserves denotation iff*

$$\llbracket \cdot \rrbracket \circ \text{compile_constraint} \equiv \llbracket \cdot \rrbracket$$

where $\llbracket \cdot \rrbracket$ maps a model or constraint onto its denotation, i.e. its logical meaning, and $\equiv$ denotes extensional function equality. Two denotations are equal iff they are logically equivalent.

3.3 Integration with MCP

The `FDSolver` type class allows us to define a generic solver `FDWrapper s` that encapsulates the mapping from the generic model to the solver-specific model.

The `FDWrapper s` is a `Solver` in the MCP framework which uses `FDConstraints` as constraints and `FDExprs` as terms.

```

1 newtype FDWrapper s a = FDWrapper { unwrapFD :: s a }
2
3 instance FDSolver s => Solver (FDWrapper s) where
4   type Constraint (FDWrapper s) = FDConstraint s
5   add c = FDWrapper $ untree $ compile_constraint c
6   ...
7   run = run . unwrapFD
8
9 instance Term (FDWrapper s) (FDTerm t) where
10   newvar = FDWrapper $ newvar >=> \x -> return $ Var x
11   ...
12
13 untree :: Solver s => Model s a -> s a
14 untree ... = ...

```

The `add` function first converts one `FDConstraint s` to a model tree for the underlying solver `s` with the `compile_constraint` function. Then, it turns this tree into a single (wrapped) monad action for the underlying solver `s`. A similar approach is used for the other solver methods. The `newvar` method of the wrapper requests a new variable from the underlying solver, and returns it wrapped in a `Var` constructor. This is why the `FDConstraint` and `FDEExpr` structures, as well as `FDWrapper` itself, are parametrized in the underlying MCP Solver `s`. Finally, `run` unwraps the encapsulated monad action and runs it.

4 The Overton Instance

The Overton FD solver is a basic finite domain solver in Haskell. It has been adapted from an implementation by David Overton⁵ to instantiate the FD-MCP framework.

Firstly, a `Solver` instance incorporates the Overton FD solver in the general MCP framework:

```

instance Solver OvertonFD where
  type Constraint OvertonFD = OConstraint
  add c          = addFD c
  run p          = runFD p
  ...

instance Term OvertonFD FDVar where ...

```

where `FDVar` is the type of terms and `OConstraint` represents the supported constraints:

```

data OConstraint
= OHasValue FDVar Int      -- [[OHasValue v c]] ≡ [[v]] = c

```

⁵ <http://overtond.blogspot.com/2008/07/pre.html>

OSame FVar FVar	-- $\llbracket \text{OSame } x \ y \rrbracket \equiv \llbracket x \rrbracket = \llbracket y \rrbracket$
ODiff FVar FVar	-- $\llbracket \text{ODiff } x \ y \rrbracket \equiv \llbracket x \rrbracket \neq \llbracket y \rrbracket$
OLess FVar FVar	-- $\llbracket \text{OLess } x \ y \rrbracket \equiv \llbracket x \rrbracket < \llbracket y \rrbracket$
OAdd FVar FVar FVar	-- $\llbracket \text{OAdd } x \ y \ z \rrbracket \equiv \llbracket x \rrbracket + \llbracket y \rrbracket = \llbracket z \rrbracket$
OSub FVar FVar FVar	-- $\llbracket \text{OSub } x \ y \ z \rrbracket \equiv \llbracket x \rrbracket - \llbracket y \rrbracket = \llbracket z \rrbracket$
OMult FVar FVar FVar	-- $\llbracket \text{OMult } x \ y \ z \rrbracket \equiv \llbracket x \rrbracket * \llbracket y \rrbracket = \llbracket z \rrbracket$

The FDSolver instance also enables the FD modeling language:

```
instance FDSolver OvertonFD where
  type FDTerm OvertonFD = FVar
  compile_constraint = convert
```

Note that the `OvertonFD` solver's term language is much more restricted than that of the generic FD-MCP modeling language: it only provides single variables (`FVar`), compared to the more extensive `FDEExpr`-expressions. The translation `convert` overcomes this syntactic mismatch.

```
convert :: FConstraint OvertonFD -> Model OvertonFD ()
convert (Less a b) = do va <- decomp a
                      vb <- decomp b
                      addC $ OLess va vb
convert ...

decomp :: FDEExpr OvertonFD -> Model OvertonFD FVar
decomp (Var f) = return f
decomp (Const i) = exists $ \v -> addC (OHasValue v i) /\ return v
decomp (Plus a b) = do
  va <- decomp a
  vb <- decomp b
  exists $ \v -> addC (OAdd va vb v) /\ return v
decomp ...
```

The auxiliary function `decomp` decomposes a compound `FDEExpr` into a model tree of `OConstraints` over `FVars`, by using the `addC` helper function to add constraints to the solver. At the same time it introduces a fresh constraint variable for each subexpression. This allows using the compact modeling language of the FD framework, although the `OvertonFD` solver only provides a much more primitive language.

Note that `compile_constraint` conforms to the law given in Definition 1. For example, assume an input expression `Plus a b`. Its denotation is $\llbracket a \rrbracket + \llbracket b \rrbracket$. Running `decomp` on it, results in an output expression `v` combined with the additional constraint `OAdd va vb v`, where `va` and `vb` are the result of applying `decomp` on `a` and `b` recursively. Assuming by induction that $\llbracket va \rrbracket \equiv \llbracket a \rrbracket$ and $\llbracket vb \rrbracket \equiv \llbracket b \rrbracket$, $\llbracket \text{OAdd } va \ vb \ v \rrbracket \equiv \llbracket v \rrbracket = \llbracket va \rrbracket + \llbracket vb \rrbracket$ leads to $\llbracket v \rrbracket \equiv \llbracket a \rrbracket + \llbracket b \rrbracket$.

Example 2. The translation of the the sum and product constraints from the FD model given in Figure 2, is shown in Figure 3. Both constraints are translated

to a combination of `NewVar` and `Add` nodes. All subexpressions in the original FD model have been decomposed into an introduction of a new variable and an additional constraint. The variables `a`, `b`, `c`, `d` refer to the original constraint variables, while `i1`, `i2`, `i3` are auxiliary variables, introduced by the translation.

5 The Gecode Instance

Rather than to run an FD-MCP model on a solver in Haskell, this section shows how to run it on a solver implemented in C++, namely Gecode. This approach illustrates three practical advantages of FD-MCP. Firstly, the framework is able to reuse existing FD solvers implemented in other programming languages. Secondly, this approach benefits from the performance characteristics of a C++ implementation. Thirdly, the high-level modeling language of the framework becomes available for low-level solvers.

The approach consists of three layers:

1. The FD-MCP solver-independent modeling language embedded in Haskell, presented earlier in Section 3.
2. The `Gecode` solver abstraction in Haskell.
3. The Gecode library in C++.

This section contributes the second layer, as well as the mappings from the first layer and to the third layer.

5.1 The Intermediate Layer: the Gecode Solver

The `Gecode` solver is a Haskell abstraction of the actual Gecode C++ solver. It defines the syntax of constraints and terms understood by Gecode:

```
instance Term Gecode IntTerm where ...

instance Solver Gecode where
  type Constraint Gecode = GConstraint
  ...
```

Gecode terms are either integer variables or integer constants:

```
data IntTerm
= IntVar Int      -- reference to variable with given ID
| IntConst Int    -- constant integer
```

The language of constraints is richer than that of the Haskell constraint solver:

```
data GConstraint
= CRel IntTerm GOp IntTerm    --  $\llbracket \text{CRel } x \text{ op } y \rrbracket \equiv \llbracket x \rrbracket \llbracket \text{op} \rrbracket \llbracket y \rrbracket$ 
| CDom IntTerm Int Int        --  $\llbracket \text{CDom } x \text{ y } z \rrbracket \equiv \llbracket x \rrbracket \in \{y, \dots, z\}$ 
| CMult IntTerm IntTerm IntTerm --  $\llbracket \text{CMult } x \text{ y } z \rrbracket \equiv \llbracket x \rrbracket * \llbracket y \rrbracket = \llbracket z \rrbracket$ 
```

```

| CallDiff [IntTerm]
    -- [[CallDiff [x1, ..., xn]]] ≡  $\bigwedge_{i \neq j} x_i \neq x_j$ 
| CLinear [(IntTerm, Int)] GOper Int
    -- [[CLinear [(x1, a1), ...] op b]] ≡  $(\sum_i a_i * \llbracket x_i \rrbracket) \llbracket op \rrbracket c$ 

data GOper
= GEqual    -- [[GEqual]] ≡ =
| GDiff     -- [[GDiff]]  ≡ ≠
| GLess     -- [[GLess]]  ≡ <

```

Note in particular the support for linear constraints `CLinear` and the all-different global constraint `CallDiff`.

5.2 From Generic FD Model to Gecode-Specific Model

In order to convert generic FD models to Gecode specific ones, the `Gecode` solver implements the `FDSolver` type class:

```

instance FDSolver Gecode where
  type FDTerm Gecode = IntTerm
  compile_constraint = mixin (linear_compile <@> basic_compile)

```

The compilation of constraints mixes two strategies. First, `linear_compile` attempts to map the `FDConstraint` onto a single linear `Gecode` constraint. If this fails, `basic_compile` decomposes the constraint, and the overall strategy is applied recursively to the decomposed parts.

Example 3. Figure 3 shows the translation of the sum and product constraints from the grocery model. The result for the product constraint is comparable to that for the Haskell solver. The sum constraint however, is converted to a single `CLinear` constraint, without any decomposition.

5.3 From Haskell to C++

The final step consists of mapping the `Gecode` solver abstraction in Haskell to the actual Gecode library in C++. It follows a code generation approach. The `gecode_compile :: Model Gecode a -> String` function transforms a `Gecode` model into the equivalent C++ source code that calls the appropriate Gecode library functions.

5.4 Disjunctive Models

Compilation of disjunctive models to Gecode is not straightforward. While general disjunction is a primitive in the MCP modeling language, it is not in Gecode.

Currently only a default compilation scheme is implemented which relies on Gecode's `when` co-routine: `when(b, ft, ff)` causes an invocation of the C++ function `ft` when the variable `b` becomes fixed to `true`, and of `ff` when it becomes

<pre>Same (Plus a (Plus b (Plus c d))) (Const 711)</pre>	<pre>Same (Mult a (Mult b (Mult c d))) (Const 711000000)</pre>
<pre>NewVar \$ \i1 -> Add (CAdd c d i1) \$ NewVar \$ \i2 -> Add (CAdd b i1 i2) \$ NewVar \$ \i3 -> Add (CAdd a i2 i3) \$ Add (CHasValue i3 711) \$ Return ()</pre>	<pre>NewVar \$ \i1 -> Add (CMult c d i1) \$ NewVar \$ \i2 -> Add (CMult b i1 i2) \$ NewVar \$ \i3 -> Add (CMult a i2 i3) \$ Add (CHasValue i3 711000000) \$ Return ()</pre>
<pre>Add (CLinear [(a,1),(b,1),(c,1),(d,1)] GEqual 711) \$ Return ()</pre>	<pre>NewVar \$ \i1 -> Add (CMult c d i1) \$ NewVar \$ \i2 -> Add (CMult b i1 i2) \$ NewVar \$ \i3 -> Add (CMult a i2 i3) \$ Add (CRel i3 GEqual 711000000) \$ Return ()</pre>

Fig. 3. Translation of the sum (left) and product (right) constraints in the grocery example for the built-in Haskell solver (top) and the Gecode solver (bottom).

false. In the future, particular cases will be compiled to the more efficient reified constraints. Gecode does however not have reified versions of all supported constraints.

Figure 4 shows the translation of this model:

```
demo = exists $ \a -> a @= 1 \/ a @= 2
```

or alternatively, its resulting model tree:

```
demo = NewVar $ \a ->
  Try
    (Add (Same a (Const 1)) $
      Return ())
    (Add (Same a (Const 2)) $
      Return ())
```

Line 2 introduces a branch-selection array of booleans *i*, here with size 1. Lines 4–10 show the new constructor. It posts an additional branching over *i* prior to posting the real branching. The actual constraints are posted in a (here root) node function `node`, shown in lines 11–14. Worth noticing is line 13 which posts the `when` constraint, passing references to functions `nodeL` and `nodeR`. The rest of the code (lines 15–22) shows the subnode functions. Both only post a single real constraint in this case.

More generally, the compilation introduces an array of boolean variables, one for each level in the disjunctive tree. For every branch and leaf a specific function

```

1  ...
2  BoolVarArray i;
3  ...
4  HaskellProg() : v0(*this,1,2), i(*this,1,0,1) {
5      node(*this);
6      branch(home, p->i, INT_VAR_SIZE_MIN, INT_VAL_MIN);
7      IntVarArgs bIntVar(1);
8      bIntVar[0]=p->v0;
9      branch(home, bIntVar, INT_VAR_SIZE_MIN, INT_VAL_MIN);
10 }
11 static void node(Space &home) {
12     HaskellProg *p = (HaskellProg *)(&home);
13     when(home, p->i[0], &nodeR, &nodeL);
14 }
15 static void nodeL(Space &home) {
16     HaskellProg *p = (HaskellProg *)(&home);
17     rel(home, p->v0, IRT_EQ, 1);
18 }
19 static void nodeR(Space &home) {
20     HaskellProg *p = (HaskellProg *)(&home);
21     rel(home, p->v0, IRT_EQ, 2);
22 }
23 ...

```

Fig. 4. Gecode C++ code generated for the disjunctive demo model

is generated in the output code. Functions for non-leaf nodes post a **when** constraint, passing pointers to the functions corresponding to its two child branches. Functions for leaf-nodes above the lowest tree depth post a dummy constraint over the remaining branch-selection variables, to prevent useless search over them. These functions contain all postings of constraints. The main function first calls the function for the root node, and proceeds by posting a branching over the branch-selection array and one over the model variables. That way, search first iterates over all branches of the disjunctive tree, storing and restoring the part of the internal state that is common to multiple branches.

5.5 Implicit Constraint Variables

As stated in Section 3.2, and shown by previous examples, the `compile_constraint` function may generate a complete model tree (without disjunctions) for a single constraint. Such a tree may introduce auxiliary constraint variables, not part of the original model. These automatically introduced, or *implicit*, constraint variables raise three concerns:

1. The programmer is not interested in these variables since they don't exist in the original model, and would like to have them excluded from the produced results.
2. Many FD solvers—including Gecode—expect an initial domain for their constraint variables. In the absence of an initial domain, propagators may diverge or take altogether too much time. The implicit variables pose a problem because the programmer has obviously not provided them with an initial domain.

3. Gecode frees variables that are no longer needed. Knowing which variables are not part of a solution allows generating code that does not keep them longer than necessary.

Thus the **Gecode** solver statically infers the lower and upper bounds of variables, using the posted constraints and the earlier bounds of the variables referenced therein.

The inference is comparable to dynamic constraint propagation, except for avoiding unbounded computations (e.g. variable ranges are represented as a lower and upper bound only). In contrast, the inference combines information across disjunctions in the search tree.

Finally, note that the inference is not restricted to the implicit variables. It also applies to the explicit constraint variables, and attempts to improve programmer-supplied initial domains.

6 Evaluation

Five existing benchmarks for Gecode were used and ported to FD-MCP. Table 1 shows the results. The columns show respectively the name of the benchmark, the problem size (if any), and the results for 1) the original Gecode benchmarks, 2) the Haskell part of the MCP benchmarks and 3) the C++ part of the MCP benchmarks. The results for each stage consist of the number of source code lines, the compilation time and the run time. Lines of code are measured using SLOccount⁶, the timings are average CPU times over multiple runs.⁷

The table shows that the MCP programs are about half as long as the original Gecode benchmarks, while the generated C++ code is much longer and grows with the problem size. However, this does not result in lower performance.

7 Related and Future Work

There is wide range of CP systems and languages. For lack of space we only mention a few. We classify them according to the distinction made in Section 1. A more extensive overview of related work can be found in [8].

Stand-alone modeling languages Zinc [6] is a stand-alone modeling language. Model transformations and compilation processes to different constraint solver backends are implemented in a second language, Cadmium, which is based on ACD term rewriting [3].

Rules2CP [4] is another stand-alone modeling language. The compilation of Rules2CP to SICStus Prolog is also specified by rewrite rules.

⁶ <http://www.dwheeler.com/sloccount/>

⁷ Benchmarks are performed on a 64-bit Ubuntu 9.04 system using a 1.67GHz Intel® Core™2 Duo T5500 processor, with 1GiB RAM. Software versions: GHC 6.10.3, GNU G++ 4.3.3, Gecode 3.1.0.

name	size	Gecode			MCP					
		C++			Haskell			C++		
		loc	g++	run	loc	ghc	run	loc	g++	run
allinterval	10	52	2.Gs	0.045s	25	1.0s	0.038s	138	2.4s	0.049s
	15			116s			0.063s	183	2.7s	129s
grocery	-	42	2.4s	0.23s	23	0.86s	0.007s	72	2.0s	0.23s
nqueens	4	80	2.5s	0.005s	25	1.0s	0.007s	79	2.1s	0.005s
	8			0.006s			0.023s	161	3.7s	0.006s
	12			0.006s			0.058s	291	6.7s	0.006s
	18			0.006s			0.27s	576	16s	0.006s
	24			0.007s			0.95s	969	42s	0.006s
magicsquare	4	62	2.6s	0.019s	34	1.0s	0.017s	122	2.4s	0.019s
	6			0.006s			0.055s	206	3.1s	0.006s
	7			4.8s			0.062s	260	3.7s	4.8s
partition	8	74	3.1s	0.007s	39	1.2s	0.027s	151	3.3s	0.006s
	16			0.045s			0.091s	247	4.3s	0.049s
	24			1.2s			0.23s	343	5.6s	1.4s

Table 1. Benchmarks

Constraint Programming API's Two other functional programming languages which provide CP support are Alice ML⁸ and FaCiLe [1]. While Alice ML provides a run-time interface to Gecode [5], FaCiLe uses its own constraint solver in OCaml. Both provide a rather low-level and imperative API, which corresponds to the C++ API of Gecode in the case of Alice ML, and relies on side effects. Neither supports alternative backends or model transformations.

Integrations Cipriano et al. [2] translate constraint models written in both Prolog CLP(FD) and (Mini)Zinc to Gecode via an intermediate language called CNT without loop constructs. The transformation from CNT to Gecode is implemented in Haskell. In order to avoid the Gecode code blow-up, it attempts to identify loops in the unrolled CNT model. It also performs a number of simplifications in the model.

Future work In future work, we would like to improve upon Cipriano et al.'s approach to directly map loop constructs from FD-MCP to Gecode, without the need for intermediate loop unrolling. A second major challenge is to generate efficient Gecode search strategies from high-level specifications using MCP's composable search transformer infrastructure.

We plan to implement a third instance of the FD-MCP framework that acts as a runtime frontend to Gecode, and directs the search from Haskell, avoiding intermediate compilation steps.

⁸ <http://www.ps.uni-sb.de/alice>

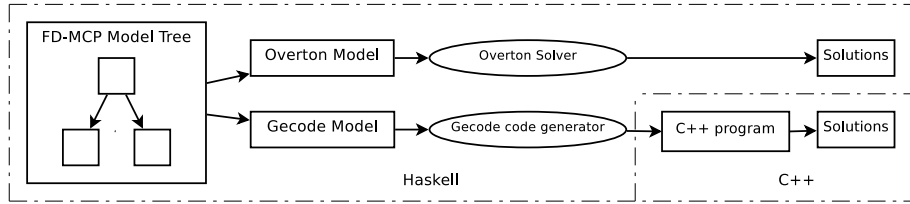


Fig. 5. FD-MCP schematic overview.

8 Conclusions

Figure 5 summarizes FD-MCP, a new layer on top of the MCP framework. FD-MCP provides a high-level declarative FD modeling language with multiple backends. Two backends are implemented: 1) a pure Haskell FD solver, and 2) a Gecode code generator. The latter brings support for high-level disjunctive models to Gecode. Benchmarks show that FD-MCP models are significantly more compact than hand-written Gecode programs, without seriously affecting performance.

Acknowledgments We are grateful to Christian Schulte and the anonymous reviewers for their helpful comments.

References

1. N. Barnier. *Application de la programmation par contraintes à des problèmes de gestion du trafic aérien*. PhD thesis, Institut National Polytechnique de Toulouse, December 2002. <http://www.recherche.enac.fr/opti/papers/thesis/>.
2. R. Cipriano, A. Dovier, and J. Mauro. Compiling and executing declarative modeling languages to Gecode. In M. G. de la Banda and E. Pontelli, editors, *ICLP*, volume 5366 of *LNCS*, pages 744–748, 2008.
3. G. J. Duck, P. J. Stuckey, and S. Brand. ACD term rewriting. In S. Etalle and M. Truszczynski, editors, *ICLP*, volume 4079 of *LNCS*, pages 117–131, 2006.
4. F. Fages and J. Martin. From Rules to Constraint Programs with the Rules2CP Modelling Language. In *Recent Advances in Constraints*, LNAI, 2009.
5. Gecode Team. Gecode: Generic constraint development environment, 2006. Available from <http://www.gecode.org>.
6. K. Marriott et al. The design of the Zinc modelling language. *Constraints*, 13(3):229–267, 2008.
7. S. Peyton Jones et al. The Haskell 98 language and libraries: The revised report. *Journal of Functional Programming*, 13(1):0–255, Jan 2003.
8. T. Schrijvers, P. Stuckey, and P. Wadler. Monadic constraint programming. *Journal of Functional Programming*, 2009.
9. P. Wadler. Monads for functional programming. In *Advanced Functional Programming*, pages 24–52, London, UK, 1995.
10. P. Wuille and T. Schrijvers. The FD-MCP framework. Report CW 562, Department of Computer Science, K.U.Leuven, August 2009.